

claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf\_c81dbe7a-7ff\agent-a4ecf3c60d3999718.jsonl`
- SHA-256: `173d1921fd073498e0f5994edad0180b8859a23540cefdf3e9b4eddca782c549`
- Source modified: `2026-06-25T17:01:45+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf\_c81dbe7a-7ff`
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`

Transcript

1. user (2026-06-25T16:56:59.173Z)

Adversarially verify this PR-review finding for PR #380 (branch o1-p11/gemini-remote-file-gc is checked out; read the real code with `git diff origin/master...HEAD` + Read the files). Be skeptical ? default to `isReal=false` / `adjustedSeverity=INVALID` if the finding is wrong, already handled by the code, or a non-issue. Do NOT re-run CI.

Finding [MEDIUM] Worst-case legitimate single-call lifetime (~40 min) leaves only ~20 min of margin under the 1h grace ? and the relevant timeouts are silent constants
Location: `backend/providers/gemini.py:132` (`process_policy max_wait_sec=1200`) and `:166` (`gen_policy op_timeout_sec=180, max_retries=5, backoff_base_sec=3.0`)
Detail: `create_time` is stamped at `files.upload` (RFC3339 creation time), and the file is needed all the way until `_safe_delete` at the END of `generate` (`gemini.py:186`). The maximum legitimate lifetime of a SINGLE in-flight file = `upload retries (~6s) + processing poll cap (max_wait_sec=1200s = 20 min) + generate (6 attempts x op_timeout_sec 180s = 1080s, plus exponential backoff 3*(1+2+4+8+16)=93s) ? 1173s. Total ? 2379s ? 39.6 min.` So a SINGLE upload that hits the worst of processing + 503-burst retries legitimately lives ~40 min ? under 1h, so a single process is safe today. BUT the margin is only ~20 min and is invisible: the grace default (3600s) lives in `gemini.py` while the timeouts that determine the floor live in two other constants in the same file with no comment linking them. If anyone later raises `max_wait_sec` or `max_retries` (e.g. for flakier capacity) without touching `grace_sec`, the worst-case lifetime can silently cross 1h and a SAME-PROCESS-restart sweep (crash + immediate relaunch while Google still has the file from a job that was killed but whose file Google keeps) could delete a file another freshly-started job is about to/again process. The coupling is real but undocumented. Suggested fix: Add a comment/assert tying `grace_sec` to the upstream timeouts: `grace_sec` MUST exceed `process_policy.max_wait_sec + (gen_policy worst-case)`. Better, derive a documented floor (e.g. note '1h > ~40min worst single-call lifetime' explicitly in the docstring) so a future timeout bump triggers a reviewer to revisit grace. Consider bumping the default to e.g. 2h to restore comfortable margin, since orphans are rare and the cost of waiting longer to reclaim is just a few extra hours of Google-side storage (files auto-expire at 48h anyway).

Return `isReal` + `adjustedSeverity` (INVALID if not real) + one-paragraph reasoning grounded in the actual code.

2. assistant (2026-06-25T16:57:01.938Z)

I'll verify this finding by reading the actual code in PR #380.

3. user (2026-06-25T16:57:03.625Z)

o1-p11/gemini-remote-file-gc
f00b71e feat(01/P11): Gemini remote-file GC ? startup orphan sweep
86705c1 chore: ruff format `gemini_refine.py` + `trajectory_hybrid.py` (#379)
b72ac85 fix(B2/P17): surface Gemini provider failures in all callers (#378)

4. user (2026-06-25T16:57:04.129Z)

```
backend/main.py | 25 ++++++++
backend/providers/gemini.py | 43 ++++++
tests/test_provider_gemini.py | 104 ++++++
3 files changed, 172 insertions(+)
```

5. assistant (2026-06-25T16:57:05.957Z)

Let me read the actual code changes.

6. user (2026-06-25T16:57:06.900Z)

```
diff --git a/backend/main.py b/backend/main.py
index 5561cee..ea5a875 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -64,6 +64,7 @@ from backend.pipeline.segmenters.base import segmenter_registry
 from backend.pipeline.stitcher.compiler import VideoCompiler
 from backend.pipeline.stitcher.watermark_i18n import localize_watermark
 from backend.pipeline.validators.base import registry
+from backend.providers import provider_registry
 from backend.reporting import emit_indexer_report
 from backend.results import Failure # standardized error/result contract
 from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
@@ -1184,6 +1185,25 @@ def _enforce_startup_or_exit(config) -> None:
     sys.exit(1)

+def _maybe_sweep_gemini_orphans(config) -> None:
+    """01/P11: best-effort sweep of Gemini File-API uploads orphaned by a previous crashed run.
+    A no-op unless the configured provider is Gemini AND its API key is present. Never raises ?
+    a failed sweep is logged and swallowed so it can't block startup. Extracted for unit
+    tests."""
+    try:
+        provider_name = getattr(config, "ai_provider", "")
+        if provider_name != "gemini":
+            return
+        api_key = os.environ.get("GEMINI_API_KEY")
+        if not api_key:
+            return
+        provider = provider_registry.get("gemini", api_key=api_key, model_name=config.ai_model)
+        deleted = provider.cleanup_orphaned_files()
+        if deleted:
+            logger.info("[STARTUP] Gemini orphan sweep reclaimed %d remote file(s).", deleted)
+    except Exception as e:
+        logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal): %s", e)
+
+def main():
+    parser = argparse.ArgumentParser(description="Sports Highlight Indexer Orchestrator")
+    parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
@@ -1319,6 +1339,11 @@ def main():
     if swept:
         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ? failed.")

+    # 01/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-call
+    # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a clip on
+    # Google's servers forever). Best-effort, video-only, grace-gated; never blocks startup.
+    _maybe_sweep_gemini_orphans(global_config)
+
+    # CLI processing mode (no web UI needed). --app/--server always run the server, never CLI
+    single-shot.
+    if args.video_path and not args.server and not args.app:
+        print(f"[CLI] Processing video file: {args.video_path}")
diff --git a/backend/providers/gemini.py b/backend/providers/gemini.py
index 2ea0834..428bca8 100644
```

```

--- a/backend/providers/gemini.py
+++ b/backend/providers/gemini.py
@@ -2,6 +2,7 @@ import json
import logging
import os
import time
+from datetime import datetime, timedelta, timezone
from typing import Tuple

from google import genai
@@ -40,6 +41,48 @@ class GeminiProvider(AIVideoProvider):
    except Exception:
        pass

+
+ def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
+     """Sweep and delete video files orphaned on the Gemini File API by a previous
+     crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
+     leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
+     that reach a ``finally``/except).
+
+     Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND whose
+     creation is older than ``grace_sec`` (so we never race a concurrent worker mid-upload).
+     Never raises; returns the count deleted. Intended to run once at startup.
+
+     Args:
+         grace_sec: only delete files older than this (default 1h). Protects a file a
+         live worker may still be processing.
+     """
+     cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
+     deleted = 0
+     try:
+         files = list(self.client.files.list())
+     except Exception as e:
+         logger.warning("[Gemini GC] failed to list remote files (skipping sweep): %s", e)
+         return 0
+     for f in files:
+         try:
+             mime = getattr(f, "mime_type", "") or ""
+             if not mime.startswith("video/"):
+                 continue
+             # f.create_time is an RFC3339 string on the File API.
+             ct = getattr(f, "create_time", None)
+             if ct is None:
+                 # No timestamp ? can't prove it's stale; leave it (safe default).
+                 continue
+             created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
+             if created > cutoff:
+                 continue # too recent ? a live worker may own it
+             self.client.files.delete(name=f.name)
+             deleted += 1
+         except Exception as e:
+             logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
+     if deleted:
+         logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
+ deleted, grace_sec)
+     return deleted
+
+
+ def analyze_video(self, video_path: str, prompt: str,
+                 chunk_duration: float = 0.0,
+                 label: str = "") -> Tuple[list, dict]:

```

7. assistant (2026-06-25T16:57:09.144Z)

Now let me read the full gemini.py to verify the timeout constants the finding references.

8. user (2026-06-25T16:57:09.935Z)

```
1     import json
2     import logging
3     import os
4     import time
5     from datetime import datetime, timedelta, timezone
6     from typing import Tuple
7
8     from google import genai
9     from google.genai import types
10
11     from backend.infrastructure.execution_policy import (
12         ExecutionPolicy,
13         PolicyTimeout,
14         poll_until,
15         run_bounded,
16     )
17     from backend.providers.base import AIVideoProvider
18
19     logger = logging.getLogger(__name__)
20
21     # The Gemini File API caps uploads at 2GB; stay well under it. Callers must chunk so
22     # each uploaded clip is below this. Oversize uploads are refused (not silently
23     attempted).
24     MAX_UPLOAD_MB = 500
25
26     class GeminiProvider(AIVideoProvider):
27         def __init__(self, api_key: str, model_name: str):
28             super().__init__(api_key, model_name)
29             # Initialize client lazily to avoid connection/auth issues during class
30             instantiation
31             self._client = None
32
33         @property
34         def client(self):
35             if self._client is None:
36                 self._client = genai.Client(api_key=self.api_key)
37             return self._client
38
39         def _safe_delete(self, video_file) -> None:
40             """Best-effort cleanup of an uploaded File (never raises)."""
41             try:
42                 self.client.files.delete(name=video_file.name)
43             except Exception:
44                 pass
45
46         def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
47             """Sweep and delete video files orphaned on the Gemini File API by a previous
48             crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
49             leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
50             that reach a ``finally``/except).
51
52             Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND
53             whose
54             creation is older than ``grace_sec`` (so we never race a concurrent worker mid-
55             upload).
56
57             Never raises; returns the count deleted. Intended to run once at startup.
58
59             Args:
60                 grace_sec: only delete files older than this (default 1h). Protects a file a
61                 live worker may still be processing.
62             """
63             cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
64             deleted = 0
```

```

60         try:
61             files = list(self.client.files.list())
62         except Exception as e:
63             logger.warning("[Gemini GC] failed to list remote files (skipping sweep):
64             %s", e)
65             return 0
66         for f in files:
67             try:
68                 mime = getattr(f, "mime_type", "") or ""
69                 if not mime.startswith("video/"):
70                     continue
71                 # f.create_time is an RFC3339 string on the File API.
72                 ct = getattr(f, "create_time", None)
73                 if ct is None:
74                     # No timestamp ? can't prove it's stale; leave it (safe default).
75                     continue
76                 created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
77                 if created > cutoff:
78                     continue # too recent ? a live worker may own it
79                 self.client.files.delete(name=f.name)
80                 deleted += 1
81             except Exception as e:
82                 logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
83         if deleted:
84             logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
85             deleted, grace_sec)
86         return deleted
87
88     def analyze_video(self, video_path: str, prompt: str,
89                       chunk_duration: float = 0.0,
90                       label: str = "") -> Tuple[list, dict]:
91         log_prefix = f"Gemini {label}" if label else "Gemini"
92
93         metrics: dict = {
94             "upload_time": 0.0,
95             "process_time": 0.0,
96             "gen_time": 0.0,
97         }
98
99         # 0. Refuse oversize uploads (Gemini File API limit is 2GB; we cap lower for
100         # safety/speed)
101         try:
102             size_mb = os.path.getsize(video_path) / (1024 * 1024)
103         except OSError:
104             size_mb = 0.0
105         if size_mb > MAX_UPLOAD_MB:
106             logger.error(f"[{log_prefix}] {video_path} is {size_mb:.0f}MB >
107             {MAX_UPLOAD_MB}MB cap ? "
108             f"SKIPPED WITHOUT ANALYSIS (no segments for this time range). "
109             f"Re-encode/downscale chunks (indexing.ai_chunk_scale_width) or
110             chunk smaller.")
111             metrics["error"] = f"oversize: {size_mb:.0f}MB > {MAX_UPLOAD_MB}MB cap"
112             metrics["oversize_skipped"] = True
113             return [], metrics
114
115         # 1. Upload (retry transient network failures, e.g. WinError 10053 / reset
116         # connections)
117         logger.info(f"[{log_prefix}] Uploading {video_path}...")
118         t_upload_start = time.time()
119         video_file = None
120         for attempt in range(3):
121             try:
122                 video_file = self.client.files.upload(file=video_path)
123                 metrics["upload_time"] = time.time() - t_upload_start
124                 logger.info(f"[{log_prefix}] Upload complete in

```

```

{metrics['upload_time']:.1f}s. URI: {video_file.uri}")
119         break
120     except Exception as e:
121         logger.warning(f"[{log_prefix}] Upload attempt {attempt + 1}/3 failed:
{e}")
122         if attempt < 2:
123             time.sleep(2 * (attempt + 1))
124     if video_file is None:
125         logger.error(f"[{log_prefix}] Upload failed after 3 attempts.")
126         metrics["error"] = "upload failed after 3 attempts"
127         return [], metrics
128
129     # 2. Wait for processing on Google servers ? a BOUNDED poll (a stuck file must
not hang
130     # the worker forever). Per-poll logs at DEBUG (no INFO spam); see
EXECUTION_POLICY.md.
131     t_process_start = time.time()
132     process_policy = ExecutionPolicy(poll_every_n_sec=10.0, max_wait_sec=1200.0) #
20 min cap
133
134     def _processed():
135         nonlocal video_file
136         video_file = self.client.files.get(name=video_file.name)
137         return None if video_file.state.name == "PROCESSING" else video_file
138
139     try:
140         video_file = poll_until(_processed, process_policy,
141                                label=f"[{log_prefix}] processing", logger=logger)
142     except PolicyTimeout:
143         logger.error(f"[{log_prefix}] Processing exceeded
{process_policy.max_wait_sec:.0f}s ? giving up.")
144         self._safe_delete(video_file)
145         metrics["error"] = f"processing timed out after
{process_policy.max_wait_sec:.0f}s"
146         return [], metrics
147     except Exception as e:
148         logger.error(f"[{log_prefix}] Status check failed: {e}")
149         self._safe_delete(video_file)
150         metrics["error"] = f"status check failed: {e}"
151         return [], metrics
152     if video_file.state.name == "FAILED":
153         logger.error(f"[{log_prefix}] Video processing failed:
{video_file.error.message}")
154         self._safe_delete(video_file)
155         metrics["error"] = f"video processing failed: {video_file.error.message}"
156         return [], metrics
157
158     metrics["process_time"] = time.time() - t_process_start
159     logger.info(f"[{log_prefix}] Processing finished in
{metrics['process_time']:.1f}s.")
160
161     # 3. Call Gemini under a HARD per-attempt timeout (op_timeout_sec ? the hang-
killer that was
162     # missing: a generate() that never returns previously blocked forever) + retry
the 503/network
163     # FAILURE waves (capacity 503s on the flagship flash come in bursts; ~6
attempts).
164     logger.info(f"[{log_prefix}] Submitting multimodal request to
{self.model_name}...")
165     t_gen_start = time.time()
166     gen_policy = ExecutionPolicy(op_timeout_sec=180.0, max_retries=5,
backoff_base_sec=3.0)
167     raw_text = None
168
169     def _generate():

```

```

170         return self.client.models.generate_content(
171             model=self.model_name,
172             contents=[video_file, prompt],
173             config=types.GenerateContentConfig(
174                 response_mime_type="application/json", temperature=0.0),
175         )
176
177     try:
178         response = run_bounded(_generate, gen_policy, label=f"{log_prefix}
generate", logger=logger)
179         raw_text = response.text
180         metrics["gen_time"] = time.time() - t_gen_start
181         logger.info(f"[{log_prefix}] Response received in
{metrics['gen_time']:.1f}s:\n{raw_text}")
182     except Exception as e:                                     # PolicyTimeout (hang) or the last 503 ?
both end here
183         logger.error(f"[{log_prefix}] generate gave up after retries: {e}")
184         metrics["error"] = f"generate failed after retries: {e}"
185         metrics["generate_failed"] = True
186         self._safe_delete(video_file)
187
188     if not raw_text:
189         return [], metrics
190
191     # 4. Parse JSON
192     try:
193         segments = json.loads(raw_text)
194         if not isinstance(segments, list):
195             logger.error(f"[{log_prefix}] Response is not a JSON array.")
196             metrics["error"] = "response is not a JSON array"
197             return [], metrics
198         return segments, metrics
199     except Exception as e:
200         logger.error(f"[{log_prefix}] JSON parse failed: {e}")
201         metrics["error"] = f"JSON parse failed: {e}"
202         return [], metrics
203

```

9. assistant (2026-06-25T16:57:13.218Z)

The timeout constants the finding references are confirmed: ``process_policy`` at line 132 (``max_wait_sec=1200.0``), ``gen_policy`` at line 166 (``op_timeout_sec=180.0, max_retries=5, backoff_base_sec=3.0``). Now let me verify how ``run_bounded`` actually applies retries and backoff, to check the finding's worst-case math.

10. user (2026-06-25T16:57:13.806Z)

```

1     """Per-job execution policy: bounded, observable waits for long/flaky ops
(EXECUTION_POLICY.md P1).
2
3     Turns an unbounded blocking call (the GX010125 Gemini hang) into a bounded one:
4     - ``run_bounded`` ? a HARD per-attempt timeout (``op_timeout_sec``, the hang-killer) +
retry-with-backoff
5     for FAILURES (an exception) *and* HANGS (no return in time). A hung attempt's worker
thread is a daemon,
6     so it is abandoned, not joined ? it never blocks the process.
7     - ``poll_until`` ? a bounded poll loop for an in-progress external op
(``poll_every_n_sec`` cadence,
8     ``max_wait_sec`` deadline).
9
10    **Testability:** ``sleep``/``monotonic`` are injected, so the whole module is unit-
tested with zero real
11    waiting (a fake clock advances instantly).
12
13    **Logging discipline** (so polling is VISIBLE but never NOISY): every poll/retry logs at

```

```

**DEBUG**; state
14     transitions (start, resolved, hung, gave-up) log at **INFO/WARNING**. INFO stays clean;
enable DEBUG on the
15     ``execution.policy`` logger to watch the poll trail.
16     """
17     from __future__ import annotations
18
19     import logging
20     import threading
21     import time as _time
22     from dataclasses import asdict, dataclass
23     from enum import Enum
24     from typing import Callable, Optional, TypeVar
25
26     T = TypeVar("T")
27
28     #: Dedicated logger so callers can dial polling visibility independently of the app's
root level.
29     LOG = logging.getLogger("execution.policy")
30
31
32     class WaitMode(str, Enum):
33         """How a job treats a long wait. ``str``-valued so it serialises straight to JSON on
the job row."""
34         WAIT_AND_RESUME = "wait_and_resume" # default: bounded wait; over budget ->
reschedule/partial (P2/P3)
35         ABORT = "abort" # one attempt; on hang/fail, stop immediately
36         BLOCK = "block" # bounded in-process blocking wait (no
reschedule)
37
38
39     class PolicyTimeout(TimeoutError):
40         """An op hung past ``op_timeout_sec`` or never became ready within
``max_wait_sec``."""
41
42
43     @dataclass(frozen=True)
44     class ExecutionPolicy:
45         """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""
46         mode: WaitMode = WaitMode.WAIT_AND_RESUME
47         poll_every_n_sec: float = 10.0
48         op_timeout_sec: float = 120.0 # HARD per-attempt deadline ? the hang-killer
that was missing
49         max_wait_sec: float = 1800.0 # total wall budget for a poll loop
50         max_retries: int = 5 # retries (=> max_retries+1 attempts) for
failures/hangs
51         backoff_base_sec: float = 3.0 # exponential: backoff_base * 2**attempt
52         on_timeout: str = "reschedule" # reschedule | partial | fail (consumed by
P2/P3)
53         resumable: bool = True
54
55         def to_dict(self) -> dict:
56             d = asdict(self)
57             d["mode"] = self.mode.value
58             return d
59
60         @classmethod
61         def from_dict(cls, d: dict) -> "ExecutionPolicy":
62             known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
63             kw = {k: v for k, v in d.items() if k in known}
64             if "mode" in kw and not isinstance(kw["mode"], WaitMode):
65                 kw["mode"] = WaitMode(kw["mode"])
66             return cls(**kw)
67

```



```

68
69 #: The sane default (also what a job gets if it declares no policy).
70 DEFAULT_POLICY = ExecutionPolicy()
71
72
73 def _backoff_sec(policy: ExecutionPolicy, attempt: int) -> float:
74     """Deterministic exponential backoff (no jitter ? reproducible tests)."""
75     return policy.backoff_base_sec * (2 ** attempt)
76
77
78 def run_bounded(fn: Callable[[], T], policy: ExecutionPolicy = DEFAULT_POLICY, *,
79               label: str = "op", logger: Optional[logging.Logger] = None,
80               sleep: Callable[[float], None] = _time.sleep) -> T:
81     """Run ``fn()`` under a hard per-attempt timeout, retrying FAILURES and HANGS with
backoff.
82
83     Returns ``fn``'s value on success. Raises ``PolicyTimeout`` if the final attempt
hung, or re-raises the
84     last exception if the final attempt failed. ``mode=ABORT`` => a single attempt (no
retries).
85     """
86     log = logger or LOG
87     attempts = 1 if policy.mode is WaitMode.ABORT else policy.max_retries + 1
88     last_exc: Optional[BaseException] = None
89     for attempt in range(attempts):
90         box: dict = {}
91         done = threading.Event()
92
93         # box/done bound as defaults (not closed over) ? the thread is started + joined
within this
94         # same iteration, but explicit binding keeps each attempt isolated and silences
B023.
95         def _runner(_box: dict = box, _done: threading.Event = done) -> None:
96             try:
97                 _box["val"] = fn()
98             except BaseException as e:          # relay ANY error (incl. timeouts) to the
caller thread
99                 _box["exc"] = e
100             finally:
101                 _done.set()
102
103         threading.Thread(target=_runner, name=f"bounded:{label}", daemon=True).start()
104         if done.wait(timeout=policy.op_timeout_sec):
105             if "exc" in box:
106                 last_exc = box["exc"]
107                 log.info("%s: attempt %d/%d failed: %s", label, attempt + 1, attempts,
last_exc)
108             else:
109                 if attempt:
110                     log.info("%s: succeeded on attempt %d/%d", label, attempt + 1,
attempts)
111                 return box["val"]          # type: ignore[return-value]
112         else:
113             last_exc = PolicyTimeout(f"{label} hung >
op_timeout_sec={policy.op_timeout_sec}s")
114             log.warning("%s: attempt %d/%d HUNG (>%.0fs) ? abandoning", label, attempt +
1, attempts,
115                         policy.op_timeout_sec)
116             if attempt < attempts - 1:
117                 wait = _backoff_sec(policy, attempt)
118                 log.debug("%s: backing off %.1fs before retry", label, wait)
119                 sleep(wait)
120             assert last_exc is not None
121             log.warning("%s: gave up after %d attempt(s): %s", label, attempts, last_exc)
122             raise last_exc

```

```

123
124
125     def poll_until(check: Callable[[], Optional[T]], policy: ExecutionPolicy =
DEFAULT_POLICY, *,
126                     label: str = "op", logger: Optional[logging.Logger] = None,
127                     sleep: Callable[[float], None] = _time.sleep,
128                     monotonic: Callable[[], float] = _time.monotonic) -> T:
129         """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
value, or raise
130         ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.
131
132         Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
INFO/WARNING.
133         """
134         log = logger or LOG
135         start = monotonic()
136         polls = 0
137         log.info("%s: waiting (poll_every=%0fs, max_wait=%0fs)",
138                 label, policy.poll_every_n_sec, policy.max_wait_sec)
139         while True:
140             result = check()
141             if result is not None:
142                 log.info("%s: ready after %d poll(s) / %0fs", label, polls, monotonic() -
start)
143                 return result
144             elapsed = monotonic() - start
145             if elapsed >= policy.max_wait_sec:
146                 log.warning("%s: still not ready after %0fs (max_wait) ? giving up", label,
elapsed)
147                 raise PolicyTimeout(f"{label} not ready within
max_wait_sec={policy.max_wait_sec}s")
148             polls += 1
149             log.debug("%s: poll %d, %0fs elapsed, not ready", label, polls, elapsed)
150             sleep(policy.poll_every_n_sec)
151

```

11. assistant (2026-06-25T16:57:22.140Z)

Now I understand the execution policy. Let me verify the finding's worst-case math against the actual code behavior.

```

**`run_bounded` worst-case for `gen_policy` (op_timeout_sec=180, max_retries=5,
backoff_base_sec=3.0):**
- attempts = max_retries + 1 = 6
- Each attempt can take up to `op_timeout_sec` = 180s before it's declared HUNG ? 6 × 180 =
1080s
- Backoff between attempts: `backoff_base * 2**attempt` for attempts 0..4 (5 backoffs, since the
last attempt has no backoff after it): 3×(2^0 + 2^1 + 2^2 + 2^3 + 2^4) = 3×(1+2+4+8+16) = 3×31 =
93s
- Total gen worst-case ? 1080 + 93 = 1173s

```

This matches the finding exactly.

```

**`poll_until` worst-case for `process_policy` (max_wait_sec=1200):** capped at 1200s. The
finding cites 1200s (20 min). Correct.

```

```

**Upload retries:** 3 attempts with `time.sleep(2*(attempt+1))` ? sleeps of 2s + 4s = 6s between
(network time excluded). Finding says ~6s. Correct.

```

Total: 6 + 1200 + 1173 = 2379s ? 39.6 min. The finding's arithmetic is accurate.

Now let me verify the key claims: (1) `create\_time` is stamped at upload, (2) the file is needed until `\_safe\_delete` at line 186, (3) grace_sec default is 3600 and lives in gemini.py, (4) the timeout constants have no comment linking them to grace_sec, (5) the same-process-restart race

scenario.

12. user (2026-06-25T16:57:22.987Z)

```
diff --git a/tests/test_provider_gemini.py b/tests/test_provider_gemini.py
index 141af81..5df2ef4 100644
--- a/tests/test_provider_gemini.py
+++ b/tests/test_provider_gemini.py
@@ -101,3 +101,107 @@ def test_gemini_provider_upload_fail():
     provider = GeminiProvider(api_key="dummy_api_key", model_name="gemini-2.5-flash")
     segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
     assert len(segments) == 0
+
+
+
+# --- 01/P11: Gemini remote-file GC (startup orphan sweep) ---
+
+def _file_mock(name, mime, create_time_iso, state="ACTIVE"):
+    f = MagicMock()
+    f.name = name
+    f.mime_type = mime
+    f.create_time = create_time_iso
+    f.state = MagicMock()
+    f.state.name = state
+    return f
+
+
+def test_cleanup_orphaned_files_deletes_stale_videos_only():
+    """Deletes video files older than the grace window; leaves non-videos and fresh files."""
+    from datetime import datetime, timedelta, timezone
+    stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
+    fresh = (datetime.now(timezone.utc) - timedelta(minutes=5)).isoformat()
+    files = [
+        _file_mock("old-vid", "video/mp4", stale),          # stale video -> deleted
+        _file_mock("fresh-vid", "video/quicktime", fresh), # fresh video -> kept (grace)
+        _file_mock("old-img", "image/png", stale),          # non-video -> kept
+        _file_mock("notts", "video/mp4", None),             # no timestamp -> kept (safe)
+    ]
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.return_value = iter(files)
+        deleted = provider.cleanup_orphaned_files(grace_sec=3600)
+    assert deleted == 1
+    provider._client.files.delete.assert_called_once_with(name="old-vid")
+
+
+def test_cleanup_orphaned_files_list_failure_is_swallowed():
+    """A list() failure must not raise ? the sweep is best-effort and must not block
+startup."""
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.side_effect = RuntimeError("network")
+        assert provider.cleanup_orphaned_files() == 0
+
+
+def test_cleanup_orphaned_files_per_file_delete_failure_is_swallowed():
+    """A delete() failure on one file must not abort the sweep."""
+    from datetime import datetime, timedelta, timezone
+    stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
+    files = [
+        _file_mock("a", "video/mp4", stale),
+        _file_mock("b", "video/mp4", stale),
+    ]
```

```

+ with patch("backend.providers.gemini.genai"):
+     provider = GeminiProvider(api_key="k", model_name="m")
+     provider._client = MagicMock()
+     provider._client.files.list.return_value = iter(files)
+     # First delete raises, second succeeds ? _safe_delete swallows it.
+     provider._client.files.delete.side_effect = [RuntimeError("boom"), None]
+     deleted = provider.cleanup_orphaned_files(grace_sec=3600)
+     assert deleted == 1 # only the second file counted
+
+
+
+## --- 01/P11: main.py startup wiring (_maybe_sweep_gemini_orphans) ---
+
+def test_maybe_sweep_skips_non_gemini_provider(monkeypatch):
+    """A non-Gemini provider must short-circuit (no client built, no sweep)."""
+    import backend.main as main_mod
+    cfg = MagicMock()
+    cfg.ai_provider = "openai"
+    called = []
+    monkeypatch.setattr(main_mod.provider_registry, "get",
+                        lambda *a, **k: called.append(k) or MagicMock())
+    monkeypatch.setenv("GEMINI_API_KEY", "k")
+    main_mod._maybe_sweep_gemini_orphans(cfg)
+    assert called == [] # provider never instantiated
+
+
+def test_maybe_sweep_skips_when_no_api_key(monkeypatch):
+    """No GEMINI_API_KEY -> no sweep (can't auth)."""
+    import backend.main as main_mod
+    cfg = MagicMock()
+    cfg.ai_provider = "gemini"
+    monkeypatch.delenv("GEMINI_API_KEY", raising=False)
+    called = []
+    monkeypatch.setattr(main_mod.provider_registry, "get",
+                        lambda *a, **k: called.append(k) or MagicMock())
+    main_mod._maybe_sweep_gemini_orphans(cfg)
+    assert called == []
+
+
+def test_maybe_sweep_runs_and_swallows_errors(monkeypatch):
+    """Gemini + key present -> sweep runs; a provider error is swallowed (never blocks
+startup)."""
+    import backend.main as main_mod
+    cfg = MagicMock()
+    cfg.ai_provider = "gemini"
+    cfg.ai_model = "gemini-2.5-flash"
+    provider = MagicMock()
+    provider.cleanup_orphaned_files.side_effect = RuntimeError("api down")
+    monkeypatch.setattr(main_mod.provider_registry, "get", lambda *a, **k: provider)
+    monkeypatch.setenv("GEMINI_API_KEY", "k")
+    # Must not raise.
+    main_mod._maybe_sweep_gemini_orphans(cfg)
+    provider.cleanup_orphaned_files.assert_called_once()

```

13. assistant (2026-06-25T16:57:27.359Z)

Now let me verify whether the finding is genuinely actionable by examining the actual concurrency model. The finding's core concern is a "same-process-restart sweep" race. Let me check how this sweep is wired relative to job execution ? whether the indexer is single-tenant/single-executor, and whether the sweep runs before or concurrently with job processing.

14. user (2026-06-25T16:57:28.525Z)

```

1320         except OSError as exc:
1321             # The lock path isn't writable (read-only dir / perms). Don't crash and don't

```

```

falsely claim a
1322         # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure
must not block a
1323         # legitimate single instance).
1324         logger.warning("[JOBS] Could not create the single-instance lock at %s.lock
(%s); proceeding "
1325                         "WITHOUT the second-instance guard.", db_path, exc)
1326         _instance_lock = None
1327     else:
1328         if _instance_lock is None:
1329             print(f"[FATAL] Another instance is already using this database ({db_path}).
Refusing to "
1330                 f"start ? a second instance would corrupt in-flight job state. Stop
the other one, or "
1331                 f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance.")
1332             sys.exit(1)
1333
1334     db_instance = Database(db_path)
1335
1336     # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
1337     # the executor is in-process. Mark them failed so pollers see a terminal state.
1338     swept = db_instance.fail_stale_jobs()
1339     if swept:
1340         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
1341
1342     # 01/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-
call
1343     # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a
clip on
1344     # Google's servers forever). Best-effort, video-only, grace-gated; never blocks
startup.
1345     _maybe_sweep_gemini_orphans(global_config)
1346
1347     # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
1348     if args.video_path and not args.server and not args.app:
1349         print(f"[CLI] Processing video file: {args.video_path}")
1350         # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
1351         # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
1352         # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
1353         storage_cfg = getattr(global_config, "storage", None)
1354         try:
1355             input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
1356         except ValueError as e:
1357             print(f"[FAIL] {e}")
1358         return
1359         if input_ref.backend == "local" and not os.path.exists(input_ref.key):

```

15. assistant (2026-06-25T16:57:37.494Z)

This is very important context. There's a ****single-instance lock**** (lines 1328-1332): a second instance is refused outright with ``sys.exit(1)`` ("Refusing to start ? a second instance would corrupt in-flight job state"). And the executor is ****in-process**** (line 1337: "the executor is in-process").

This materially undercuts the finding's race scenario. The finding posits: "a SAME-PROCESS-restart sweep (crash + immediate relaunch while Google still has the file from a job that was killed but whose file Google keeps) could delete a file another freshly-started job is about to/again process."

Let me carefully reason about whether the grace window protects against this. When the process crashes and relaunches:

1. The crashed process's jobs are marked failed (line 1338 `fail\_stale\_jobs`) ? so they don't auto-resume the same in-flight Gemini call. The killed job's file would be a genuine orphan.
2. The single-instance lock means there is exactly ONE process. The sweep runs at startup, ****before**** the server starts accepting/processing new jobs.

So the race the finding describes ? "another freshly-started job is about to/again process" the file concurrently with the sweep ? requires concurrent execution of the sweep and a live job touching the same file. Let me check whether anything could be processing during the sweep. The sweep runs synchronously at startup before the server loop. Let me confirm there's no concurrent worker/job thread active at the moment `\_maybe\_sweep\_gemini\_orphans` runs.

16. user (2026-06-25T17:00:45.496Z)

```
91:     ``backend.main`` and ``backend.api.job_worker`` into mypy quarantine.
272:# now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
273:# `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
278:from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable on
backend.main)
283:    _get_executor,
871:    _get_executor().submit(_drain_due_jobs, request.app.state.config, request.app.state.db)
1048:    _get_executor().submit(_drain_due_jobs, request.app.state.config, request.app.state.db)
1337:    # the executor is in-process. Mark them failed so pollers see a terminal state.
1443:        threading.Thread(target=_open_browser_when_ready, args=(url, url +
"api/health"),
1457:        _get_executor().submit(_drain_due_jobs, global_config, db_instance)
1464:    # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
```

17. assistant (2026-06-25T17:00:51.532Z)

Confirmed. The job executor processes jobs via `\_get\_executor().submit(\_drain\_due\_jobs, ...)` which happens either inside an API request handler (lines 871, 1048) or at line 1457 ? ****after**** the sweep at line 1345. Let me verify line 1457's position relative to the sweep.

18. user (2026-06-25T17:00:52.456Z)

```
1430         if args.app:
1431             # P2: the friendly launcher ? always localhost-only (use --server for
network exposure),
1432             # auto-pick a free port if --port is busy, and open the bundled UI once the
server is up.
1433             bind_host = "127.0.0.1"
1434             port = _resolve_app_port(bind_host, args.port)
1435             if port != args.port:
1436                 print(f"[APP] Port {args.port} is busy ? using {port} instead.")
1437             url = f"http://{bind_host}:{port}/"
1438             # The browser-open health poll sends Host: 127.0.0.1 ? it relies on loopback
staying in
1439             # the Host allowlist (the default; don't set RALLY_ALLOWED_HOSTS to a non-
loopback-only set
1440             # with --app, or the poll 400s and the browser just won't auto-open ? the
server still runs).
1441             print(f"[APP] Starting KhelSutra ? opening {url} in your browser "
1442                   f"(set RALLY_NO_BROWSER=1 to skip). Press Ctrl+C to stop.")
1443             threading.Thread(target=_open_browser_when_ready, args=(url, url +
"api/health"),
1444                             daemon=True).start()
1445         else:
1446             bind_host = _resolve_bind_host(global_config, args.host) # P0c:
config/profile-driven (default 127.0.0.1)
1447             port = args.port
1448             print(f"[SERVER] Starting local FastAPI server on
```

```

http://{bind_host}:{port}...)
1449         if bind_host not in ("127.0.0.1", "localhost", "::1"):
1450             print(f"[SERVER] ? Binding {bind_host} exposes the API on the network ?
ensure an auth "
1451                 f"gate (edge_auth_enabled / a reverse proxy) is in front, and set
"
1452                 f"RALLY_ALLOWED_HOSTS to your public host(s) (the Host-header
guard defaults to "
1453                 f"loopback-only and will 400 forwarded requests otherwise).")
1454         # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
1455         # in-process wake-timer for any parked 'waiting' job died with the old process,
so
1456         # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
1457         _get_executor().submit(_drain_due_jobs, global_config, db_instance)
1458         # Stamp shared state on the app so route handlers access it via
request.app.state
1459         # instead of the Optional module-level globals (CODE_CLEANUP_AUDIT P5 / D2).
1460         create_app(global_config, db_instance)
1461         uvicorn.run(app, host=bind_host, port=port)
1462
1463     if __name__ == "__main__":
1464         # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the

```

19. assistant (2026-06-25T17:01:16.850Z)

This confirms the ordering definitively. The sweep at line 1345 runs **before** any job executor work is submitted (line 1457) and before the server starts accepting requests (line 1461). So at the moment the sweep runs:

- There is exactly one process (single-instance lock at 1328-1332).
- No job is being processed yet (the first `\_drain\_due\_jobs` submit is at 1457, after the sweep).
- `fail\_stale\_jobs()` (1338) already marked any prior-process jobs as failed.

So the race the finding describes (sweep deleting a file "another freshly-started job is about to/again process") **cannot occur in the current code** ? the sweep completes synchronously before any job runs, and there's only one process. The grace window provides defense-in-depth for any future change to this ordering (e.g. moving to multi-tenant or out-of-process workers), but the present-day race is not real.

Now let me assess the finding's actual substance. The finding itself **explicitly concedes** that "a single process is safe today" and that the worst-case (~40 min) is "under 1h, so a single process is safe today." So the finding is NOT claiming a present-day bug. Its claim is purely about:

1. **Tight margin** (~20 min) that is **invisible/undocumented** ? the grace constant and the timeout constants live apart with no comment linking them.
2. A **latent future risk**: if someone bumps `max\_wait\_sec` or `max\_retries` without touching `grace\_sec`, the worst-case could cross 1h and enable a race.

Let me double-check the math one more time and verify the finding's characterization of the constants being "silent" (no linking comment). Looking at the diff: the docstring at line 55 says "default 1h" with no reference to the upstream timeouts. The constants at lines 132 and 166 have comments about their own purpose but nothing tying them to `grace\_sec`. The finding's characterization is accurate.

Let me also sanity-check the finding's worst-case backoff assumption against `run\_bounded`: backoff happens `if attempt < attempts - 1` (line 116), i.e. after attempts 0,1,2,3,4 (not after attempt 5, the last). So 5 backoffs: $3 \times (1+2+4+8+16) = 93s$. The finding's " $3 \times (1+2+4+8+16) = 93s$ " is exactly right.

Now let me evaluate severity. The finding correctly states there's no present-day bug. The real content is a maintainability/documentation concern (coupling between constants is undocumented) plus a latent risk. Let me consider:

- ****Is it a real concern?**** Yes, the coupling is genuine and undocumented. A future timeout bump genuinely could erode the safety margin invisibly. This is a legitimate observation.

- ****Is the risk overstated?**** Partially. Even if the worst-case lifetime crossed 1h, for the race to bite, the code would *also* need to change to run jobs concurrently with the sweep, or the sweep would need to run mid-job. With the current architecture (single instance + sweep-before-jobs + ``fail_stale_jobs``), even a >1h lifetime wouldn't create the race the finding describes during normal startup, because no job is in-flight when the sweep runs. The finding's "freshly-started job is about to/again process" assumes a concurrency that the current architecture forbids. So the race scenario requires *two* future changes, not one.

Let me verify there's no other call path to ``cleanup_orphaned_files`` that could run concurrently with jobs.

20. user (2026-06-25T17:01:17.958Z)

```
backend\providers\gemini.py:44: def cleanup_orphaned_files(self, grace_sec: int = 3600) ->
int:
tests\test_provider_gemini.py:119: def test_cleanup_orphaned_files_deletes_stale_videos_only():
tests\test_provider_gemini.py:134:         deleted =
provider.cleanup_orphaned_files(grace_sec=3600)
tests\test_provider_gemini.py:139: def test_cleanup_orphaned_files_list_failure_is_swallowed():
tests\test_provider_gemini.py:145:         assert provider.cleanup_orphaned_files() == 0
tests\test_provider_gemini.py:148: def
test_cleanup_orphaned_files_per_file_delete_failure_is_swallowed():
tests\test_provider_gemini.py:162:         deleted =
provider.cleanup_orphaned_files(grace_sec=3600)
tests\test_provider_gemini.py:202:         provider.cleanup_orphaned_files.side_effect =
RuntimeError("api down")
tests\test_provider_gemini.py:207:         provider.cleanup_orphaned_files.assert_called_once()
backend\main.py:1200:         deleted = provider.cleanup_orphaned_files()
```

21. assistant (2026-06-25T17:01:29.938Z)

Confirmed: ``cleanup_orphaned_files`` is called from exactly one place in production ?
``_maybe_sweep_gemini_orphans`` (main.py:1200), which is invoked once at startup (main.py:1345), before the executor and server start. There is no concurrent invocation path.

I now have everything needed to render a verdict. Let me summarize my assessment:

****Accuracy of the finding's facts:****

- Constants verified: ``max_wait_sec=1200`` (line 132), ``op_timeout_sec=180``, `max_retries=5``, `backoff_base_sec=3.0`` (line 166), ``grace_sec=3600`` default (line 44). ?
- Worst-case math verified against ``run_bounded`/`poll_until``: upload ~6s + processing cap 1200s + generate (6×180 + 93 backoff = 1173s) ? 2379s ? 39.6 min. ?
- ``create_time`` stamped at upload, file needed until ``_safe_delete`` at line 186. ?
- Constants are undocumented w.r.t. their coupling to ``grace_sec``. ?

****Where the finding overreaches:****

- The race scenario ("a freshly-started job is about to/again process" a file the sweep deletes) cannot occur in the current architecture: single-instance lock refuses a 2nd process, ``fail_stale_jobs`` kills prior jobs, and the sweep runs synchronously ****before**** any job executes or the server accepts requests. So even if the worst-case lifetime crossed 1h, the described race needs an *additional* architectural change (concurrent job execution during the sweep), not just a timeout bump.
- The finding itself concedes there is no present-day bug ("a single process is safe today").

So this is a ****valid maintainability/documentation observation**** (real, undocumented coupling worth a comment/assert), but ****not a present-day correctness bug****, and the latent-race severity is somewhat inflated because the current architecture provides a second, stronger guard (sweep-before-jobs + single-instance) that the finding's worst-case-crosses-1h scenario alone wouldn't defeat. This lands as a NIT ? a reasonable, low-cost documentation/defensive-coding suggestion, not a MEDIUM-severity issue.

22. user (2026-06-25T17:01:38.344Z)

Structured output provided successfully

23. assistant (2026-06-25T17:01:45.729Z)

I've completed the adversarial verification of the finding for PR #380.

****Verdict: isReal=true, adjustedSeverity=NIT****

The finding's facts and math check out, but it overstates the severity. Key findings from reading the real code:

- ****Constants confirmed**** (`backend/providers/gemini.py`): `max\_wait\_sec=1200` (line 132), `op\_timeout\_sec=180`/`max\_retries=5`/`backoff\_base\_sec=3.0` (line 166), `grace\_sec=3600` default (line 44). The ~39.6 min worst-case lifetime math is correct when traced through `run\_bounded`/`poll\_until` in `backend/infrastructure/execution\_policy.py`.
- ****The undocumented coupling is real**** ? a legitimate maintainability observation.
- ****But the headline race cannot occur in the current architecture.**** The sweep (`\_maybe\_sweep\_gemini\_orphans`, `backend/main.py:1200`) runs once at startup (line 1345), synchronously ****before**** the job executor submits any work (line 1457) and before uvicorn accepts requests (line 1461). A single-instance lock (lines 1328-1332) `sys.exit(1)`'s any second process, and `fail\_stale\_jobs()` (line 1338) already terminates prior jobs. No job is ever in-flight concurrently with the sweep, so even a >1h worst-case lifetime wouldn't enable the described race without a *separate* architectural change.

The finding itself concedes "a single process is safe today," so it's a low-cost defensive-documentation suggestion (add a comment/assert tying `grace\_sec` to the upstream timeout floor), not a MEDIUM correctness risk.